

PROJECT REPORT

PROJECT DOCUMENTATION

PROJECT TITLE	Script-Controlled ACL - Restrict Record Based On Field Value (ServiceNow Incident Management Project)
----------------------	---

TEAM MEMBERS	Dharshini S (Team Leader)
---------------------	---------------------------

TEAM LEADER	Dharshini S
--------------------	-------------

DEPARTMENT	B.Tech Information Technology
-------------------	-------------------------------

COLLEGE	Sri Ramanujar Engineering College
----------------	-----------------------------------

ACADEMIC YEAR	2026 - 2027
----------------------	-------------

GUIDE	
--------------	--

Prepared according to the prescribed project-report format.

CERTIFICATE

This is to certify that the project report entitled "**Script-Controlled ACL - Restrict Record Based On Field Value**" is a bonafide record of the work carried out by **Dharshini S (Team Leader)** and team members of the Department of Information Technology, Sri Ramanujar Engineering College, under the guidance and supervision of the faculty guide during the academic year 2026-2027.

Project Guide

Signature: _____

Date: _____

Head of the Department

Signature: _____

Date: _____

DECLARATION

We hereby declare that the project work presented in this report is our original work carried out as part of the academic requirements. The work has been completed with proper acknowledgement of the sources, tools, datasets, libraries and references used.

Place: _____

Date: _____

Team Members' Signatures

1. Dharshini S (Team Leader) _____

2. _____

3. _____

4. _____

ABSTRACT

Organisations that use ServiceNow for IT Service Management store incident records of very different sensitivity in a single table. Standard role-based access allows every user holding a common role, such as *itil*, to open any incident, including critical or confidential ones. This project addresses that gap by implementing a **script-controlled Access Control List (ACL)** that restricts access to Incident records based on the value of a field.

A server-side JavaScript condition is written inside a record-level ACL on the *incident* table. When a user opens, lists or queries a record, the script reads the field value of that record (Priority = 1 - Critical) and the identity and roles of the logged-in user, and returns *true* or *false*. Authorised users, namely administrators, incident managers and the caller or assignee of the ticket, can view the record, while all other users are denied and the record is hidden from list views. The solution was configured and validated on a ServiceNow Personal Developer Instance using user impersonation and a set of test cases.

The result is a fine-grained, maintainable and auditable security layer that protects sensitive incidents without custom UI code, improving confidentiality while keeping normal incident handling unaffected.

Keywords: ServiceNow, Incident Management, Access Control List (ACL), Script-Controlled ACL, Role-Based Access Control, Field-Level Security, ITSM

TABLE OF CONTENTS

1. INTRODUCTION	6
1.1 Project Overview	6
1.2 Purpose	6
2. IDEATION PHASE	7
2.1 Problem Statement	7
2.2 Empathy Map Canvas	7
2.3 Brainstorming	7
3. REQUIREMENT ANALYSIS	9
3.1 Customer Journey Map	9
3.2 Solution Requirement	9
3.3 Data Flow Diagram	10
3.4 Technology Stack	11
4. PROJECT DESIGN	12
4.1 Problem Solution Fit	12
4.2 Proposed Solution	12
4.3 Solution Architecture	13
5. PROJECT PLANNING & SCHEDULING	14
5.1 Project Planning	14
6. FUNCTIONAL AND PERFORMANCE TESTING	15
6.1 Performance Testing	15
7. RESULTS	16
7.1 Output Screenshots	16
8. ADVANTAGES & DISADVANTAGES	18
9. CONCLUSION	18
10. FUTURE SCOPE	18
11. APPENDIX	19
Source Code (if any)	19
Dataset Link	19
GitHub & Project Demo Link	19

1. INTRODUCTION

ServiceNow is a leading cloud platform for IT Service Management (ITSM). Its Incident Management application records, tracks and resolves unplanned interruptions to IT services. Every incident is stored as a record in the *incident* table, and each record contains fields such as number, caller, category, priority, assignment group, assigned to and state.

Security on the platform is enforced through Access Control Lists (ACLs). An ACL defines which users may perform an operation (create, read, write, delete) on a table, a record or a field. Most ACLs rely on roles, but real-world situations often need a decision that also depends on the **data inside the record**. For example, a critical incident affecting executives or security systems should not be visible to every support agent. Script-controlled ACLs solve this by allowing a short server-side script to evaluate any condition and return a true or false answer.

1.1 Project Overview

This project, "**Script-Controlled ACL - Restrict Record Based On Field Value**", implements a record-level read ACL on the Incident table of ServiceNow. The script inspects the *priority* field of the record being accessed. If the incident is Critical (Priority 1), access is granted only to administrators, users holding the custom role *incident_manager*, and the caller or assignee of that incident. For incidents of any other priority, existing role-based rules continue to apply.

The system is intended for ServiceNow administrators, security officers and service-desk teams who need to protect sensitive records. The expected outcome is that unauthorised users can neither see restricted incidents in list views and reports nor open them through a direct URL.

1.2 Purpose

The purpose of the project is to demonstrate and document how conditional, data-driven security can be added to ServiceNow without modifying the core application. The main objectives are:

- To restrict access to incident records according to the value of a chosen field.
- To allow only authorised roles and directly involved users to see restricted records.
- To keep the ACL script simple, efficient and easy to maintain.
- To validate the solution through impersonation-based test cases and document the results.

Compared with the existing situation, where any *itil* user can open any incident, the proposed approach improves confidentiality, supports compliance and reduces the risk of accidental data exposure.

2. IDEATION PHASE

The team identified the problem by studying how access works in a default ServiceNow instance and by considering the situations in which a service desk handles sensitive tickets. User-centred techniques, namely a problem statement, an empathy map and brainstorming, were used to arrive at the final solution.

2.1 Problem Statement

Target users: ServiceNow administrators, security/compliance officers, incident managers and service-desk agents.

Current difficulty: Any user with the *itil* role can open every incident, even those that are critical or confidential.

Cause: Role-based ACLs are coarse. They decide access by role only and cannot look at the value of a field in the record.

Impact: Sensitive information may be exposed to staff who have no need to know, creating confidentiality, audit and compliance risks.

How might we restrict access to individual incident records according to a field value, while keeping normal incident handling smooth?

2.2 Empathy Map Canvas

The empathy map below represents the primary user, the ServiceNow administrator / security officer responsible for protecting incident data.

SAYS	THINKS
"Not every agent should see critical tickets." "I need a rule based on the ticket data, not just roles." "Can we audit who accessed what?"	Role-based rules are too broad. A mistake could leak confidential details. There must be a way to add logic without heavy customisation.
DOES	FEELS
Reviews roles and ACLs. Tests access through impersonation. Escalates critical tickets to managers.	Concerned about data exposure. Pressured by compliance requirements. Relieved when access is controlled and predictable.
PAINS	GAINS
No field-based restriction by default. Manual workarounds are error-prone. Hard to prove restrictions to auditors.	Automatic, rule-driven protection. Centralised and easy-to-maintain control. Improved trust and audit readiness.

Figure 2.1: Empathy map of the administrator / security officer

2.3 Brainstorming

The team considered several ways to restrict records and compared them against effort, flexibility and maintainability.

Idea	Description	Decision
A. Role-only ACL	Create more roles and give each agent a different role.	Rejected - role explosion, no field awareness.
B. Data Policy / UI Policy	Hide or lock fields on the form.	Rejected - does not stop record access or list/API access.
C. Domain separation	Split data into separate domains.	Rejected - too heavy for the requirement.
D. Condition-based ACL	Use the ACL condition builder with a field value.	Partly suitable - cannot handle caller/assignee or multiple roles.
E. Script-controlled ACL	Server-side script checks field value, roles and user identity.	Selected - flexible, native, lightweight.

Idea E was preferred because it works for UI, list views, reports and APIs alike, requires no change to the core application and allows the logic to be extended easily.

3. REQUIREMENT ANALYSIS

This phase identifies what the system must do, who will use it and what environment is required to build and test it.

3.1 Customer Journey Map

The journey below follows a service-desk agent who tries to open a critical incident, and shows how the ACL changes the experience.

Stage	Need / Login	Search	Open record	Outcome	Follow-up
Actions	Logs in to ServiceNow with assigned role.	Opens incident list and filters tickets.	Clicks a critical incident or uses a direct URL.	ACL script runs; access allowed or denied.	Requests access via manager if needed.
Pain points	Unsure which tickets are visible.	Unexpected missing records.	Security message if not authorised.	May not know why access was denied.	Delay in getting access.
Emotions	Neutral	Curious	Concerned	Satisfied (authorised) / Informed (denied)	Reassured
Opportunities	Clear role guidance.	Hide restricted rows cleanly.	Friendly denial message.	Log denied attempts for audit.	Self-service access request.

Figure 3.1: Customer journey map

3.2 Solution Requirement

Functional requirements

- FR1: The system shall evaluate an ACL script each time an incident record is read.
- FR2: Critical (Priority 1) incidents shall be accessible only to administrators, incident managers, and the incident caller or assignee.
- FR3: Incidents of other priorities shall follow existing role-based access.
- FR4: Restricted records shall not appear in list views, reports or searches for unauthorised users.
- FR5: Direct URL access to a restricted record shall be blocked.

Non-functional requirements

- Security: default-deny behaviour for restricted records.
- Performance: script must be lightweight, with no database queries inside the ACL.
- Maintainability: short, commented and reusable script.
- Reliability: results must be consistent across UI, list and API access.

Hardware / software requirements

- Computer with a modern web browser (Chrome / Edge) and stable internet connection.

- ServiceNow Personal Developer Instance (PDI) with Incident Management plugin.
- Administrator access and the *security_admin* elevated role to create ACLs.

User requirements: test users with roles *itil* and *incident_manager*, and an administrator account. **Data requirements:** sample incident records with different priorities and callers (no external dataset is used).

3.3 Data Flow Diagram

The Level 0 DFD shows the system as a single process interacting with the user, the administrator and the incident table.

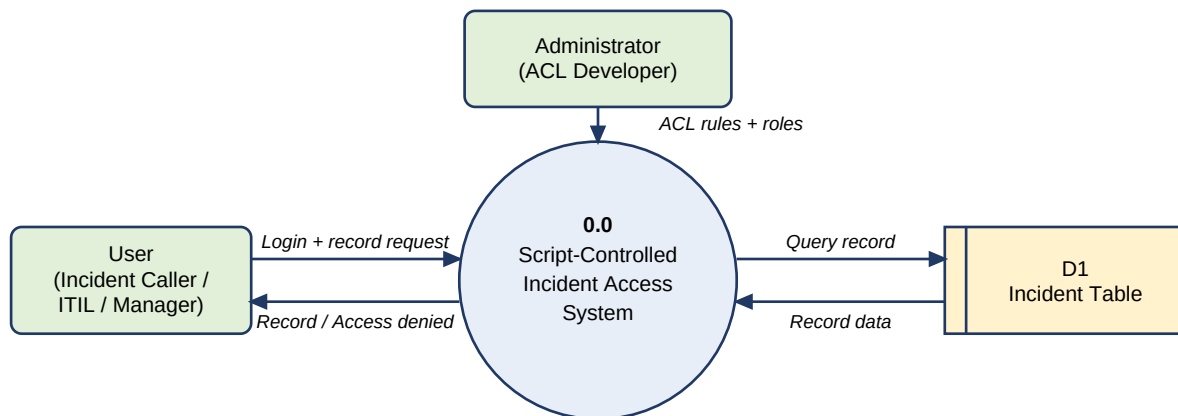
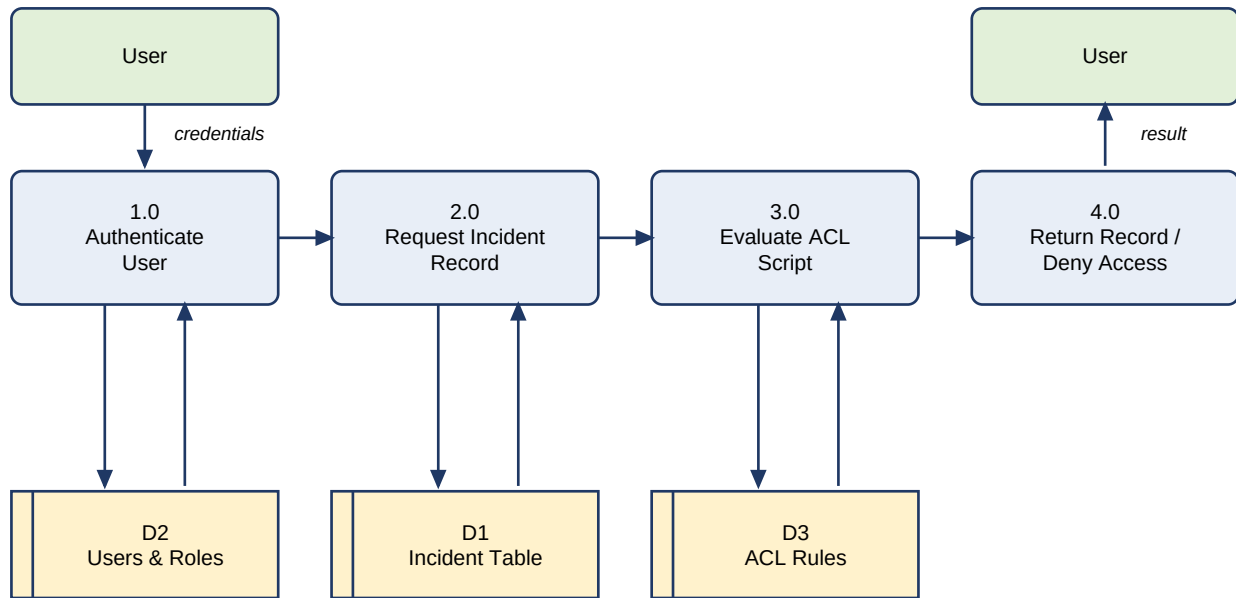


Figure 3.2: Data Flow Diagram - Level 0

The Level 1 DFD decomposes the system into four processes. **1.0 Authenticate User** verifies credentials against the Users & Roles store. **2.0 Request Incident Record** receives the open/list request and queries the Incident table. **3.0 Evaluate ACL Script** reads the ACL rule and compares the record's priority with the user's roles and identity. **4.0 Return Record / Deny Access** sends either the record data or a security message back to the user.



Arrows to/from stores = read / write of data

Figure 3.3: Data Flow Diagram - Level 1

3.4 Technology Stack

Layer	Technology	Purpose
Frontend	ServiceNow Platform UI (Incident list and form)	User interface for viewing and managing incidents
Backend	Server-side JavaScript (GlideSystem, GlideRecord API), ACL script	Application logic and access decision
Database	ServiceNow relational database (incident, sys_user, sys_user_has_role, sys_security_acl)	Data storage
Tools	Personal Developer Instance, ACL editor, User Impersonation, Security Debugger, GitHub	Development, testing and version tracking
Deployment	ServiceNow cloud instance (PDI); transferable using Update Sets	Hosting and migration

4. PROJECT DESIGN

The requirements are converted into a design that uses native ServiceNow security features so that the solution is practical, secure and easy to deploy.

4.1 Problem Solution Fit

User problem	Solution feature	Fit
Any itil user can open critical incidents	Record-level read ACL with script checking Priority	Directly blocks unauthorised access
Role-only control cannot see field values	Script reads <i>current.priority</i>	Decision uses record data
Legitimate users (manager, caller, assignee) must still work	Script allows role, caller and assignee	No disruption to valid work
Restricted tickets visible in lists and reports	ACL applies to list, report and API	Consistent hiding of records
Need for easy maintenance	Short, commented script; role-based exception	Simple to modify

4.2 Proposed Solution

The proposed system adds one custom role and one script-controlled ACL to the Incident table. It consists of the following modules and workflow:

- **Role module:** create the role *incident_manager* and assign it to authorised users.
- **Test data module:** create users and incident records with varied priorities and callers.
- **ACL module:** create a *record* ACL on table *incident*, operation *read*, with advanced script enabled.
- **Script module:** evaluate the field value and user identity and set *answer* to true or false.
- **Validation module:** impersonate each user and verify access.

Workflow: User requests record → platform finds matching ACLs → role and condition checks pass → script is executed → *answer = true* grants access, *answer = false* denies it. In ServiceNow, a user needs every matching ACL to pass, so the new rule safely adds a restriction on top of the existing ones.

Core script logic:

```
(function() {  
    answer = false;  
    // Non-critical incidents: allow (other ACLs still apply)  
    if (current.priority != 1) { answer = true; return; }  
    // Critical incidents: restrict  
    var me = gs.getUserID();  
    if (gs.hasRole('admin') || gs.hasRole('incident_manager') ||  
        current.caller_id == me || current.assigned_to == me)  
        answer = true;  
})();
```

4.3 Solution Architecture

The architecture has three layers. The **presentation layer** is the ServiceNow web interface used by end users. The **application layer** contains the Incident Management application, the script-controlled ACL engine and the roles, groups and session information. The **data layer** holds the incident, user, role mapping and ACL tables. Every request passes through the ACL engine before data is returned.

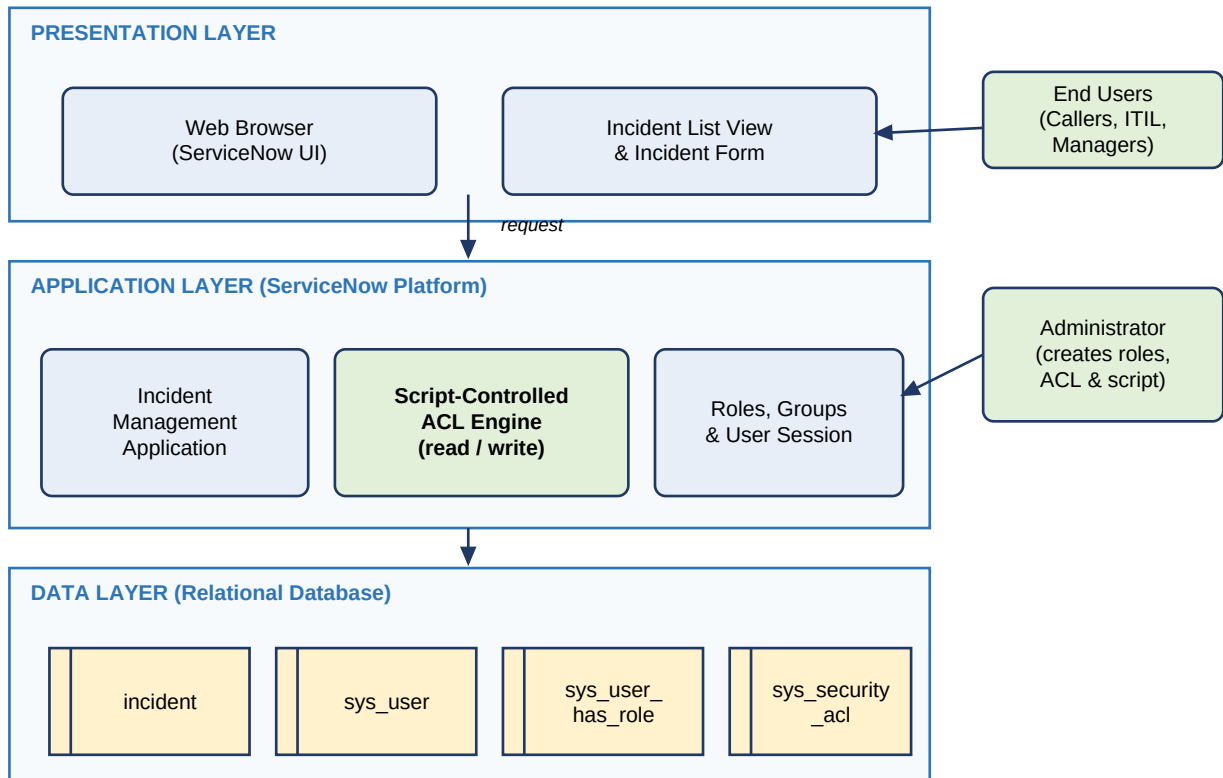


Figure 4.1: Solution architecture

5. PROJECT PLANNING & SCHEDULING

The work was divided into phases with clear responsibilities and milestones over an eight-week period.

5.1 Project Planning

Phase	Activities	Weeks	Responsibility
Problem identification	Study default access; define problem statement	1	Dharshini S
Requirement analysis	Functional / non-functional requirements, DFD	1-2	Dharshini S & team
Design	Role design, ACL logic, architecture	2-3	Dharshini S
Development	PDI setup, roles, users, test data, ACL script	3-5	Team
Testing	Impersonation tests, debugging, fixes	5-6	Team
Documentation	Report, diagrams, screenshots	6-7	Dharshini S
Demonstration	Final demo and submission	8	Team

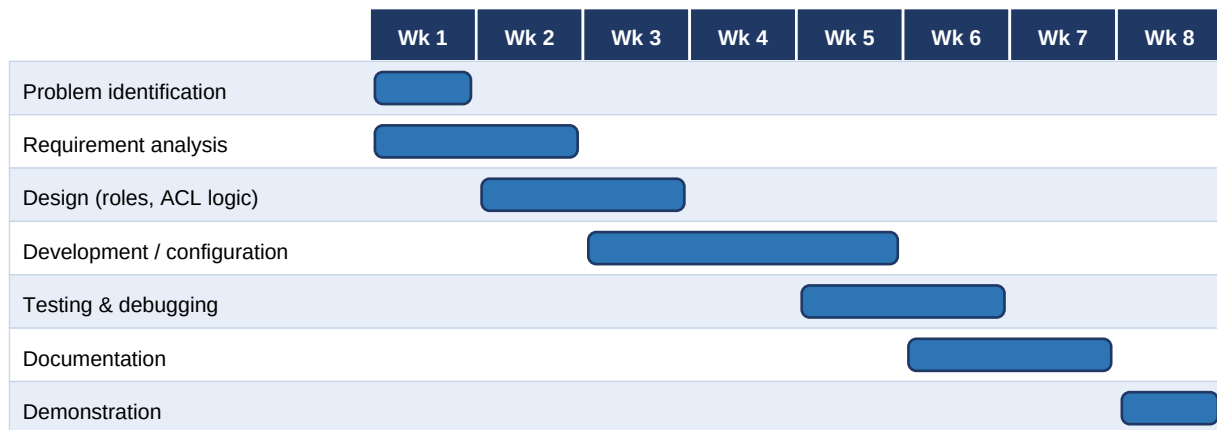


Figure 5.1: Project schedule (Gantt chart)

6. FUNCTIONAL AND PERFORMANCE TESTING

Testing was carried out on the ServiceNow instance using the *Impersonate User* feature to confirm that the ACL satisfies the requirements for each type of user.

6.1 Performance Testing

ID	Feature / Scenario	Input	Expected Output	Actual Output	Status
TC-01	Admin opens critical incident	User: admin; Priority 1	Record opens	Record opened	PASS
TC-02	Incident manager opens critical incident	Role: incident_manager; Priority 1	Record opens	Record opened	PASS
TC-03	ITIL user (not caller) opens critical incident	Role: itil; Priority 1	Access denied	Security message shown	PASS
TC-04	ITIL user opens non-critical incident	Role: itil; Priority 3	Record opens	Record opened	PASS
TC-05	Caller views own critical incident	User = caller; Priority 1	Record opens	Record opened	PASS
TC-06	Assignee views assigned critical incident	User = assigned to; Priority 1	Record opens	Record opened	PASS
TC-07	List view for restricted user	Role: itil; open Incident list	Priority 1 rows hidden	Rows hidden	PASS
TC-08	Direct URL to restricted record	sys_id link of P1 incident	Access blocked	Blocked	PASS

Observations: The ACL script uses only field comparisons and role checks and performs no database queries, so no noticeable delay was observed when opening records or loading lists. Results were consistent across form, list view and direct link access. The behaviour is repeatable for every impersonated user, which indicates good reliability. The denial message is the platform default, which is clear but could be customised in future.

7. RESULTS

The final system successfully hides and blocks critical incidents from unauthorised users while allowing administrators, incident managers, callers and assignees to work normally. The following screenshots demonstrate the working features.

7.1 Output Screenshots

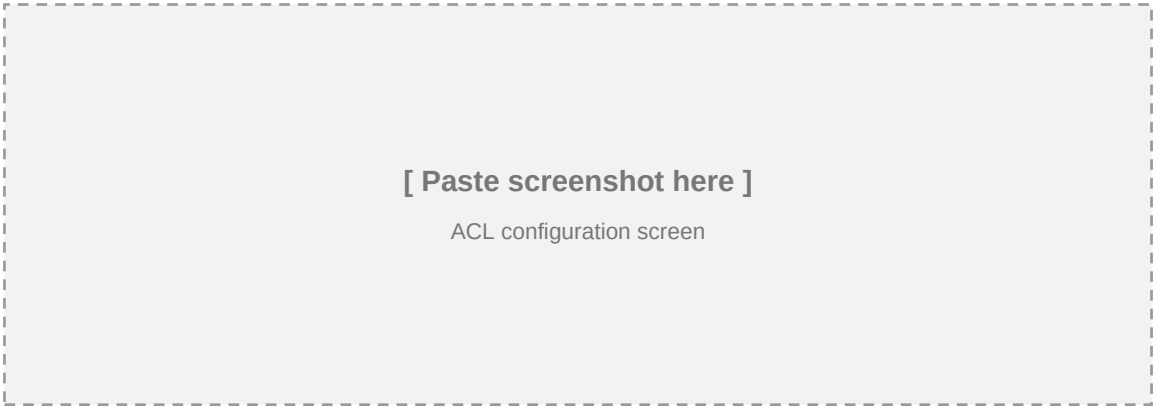


Figure 7.1: ACL configuration - table incident, operation read, type record

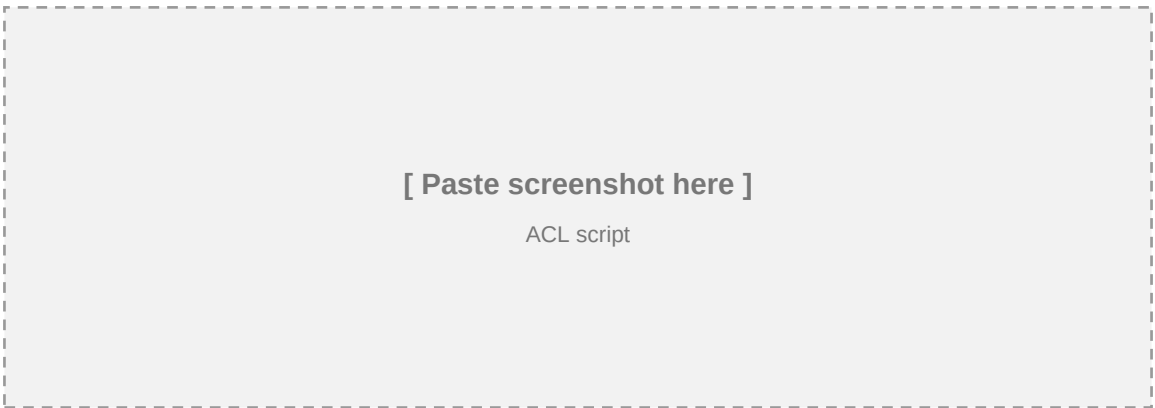


Figure 7.2: Advanced script of the ACL

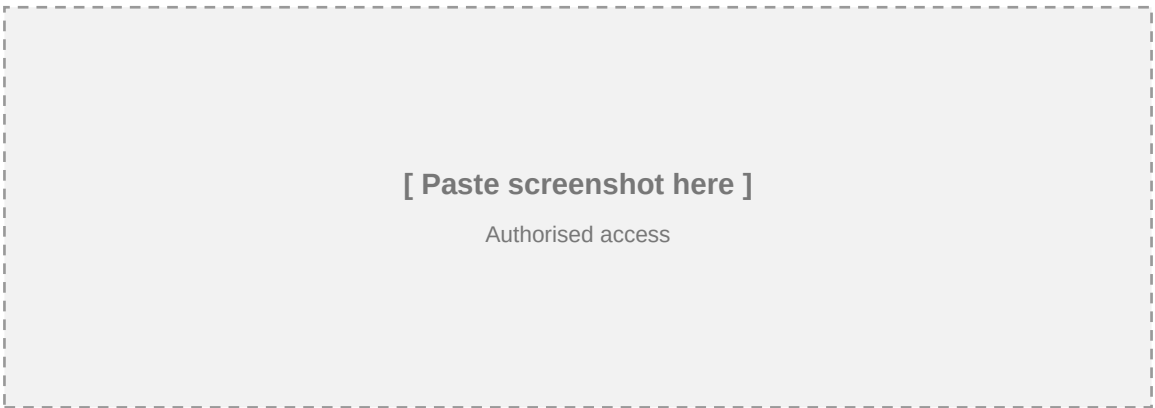


Figure 7.3: Administrator / incident manager viewing a critical incident

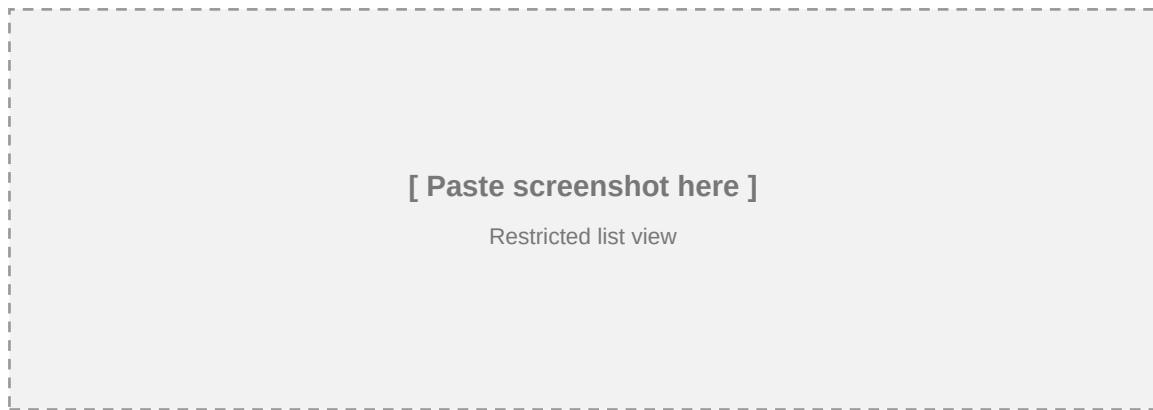


Figure 7.4: ITIL user - critical incidents hidden in list view

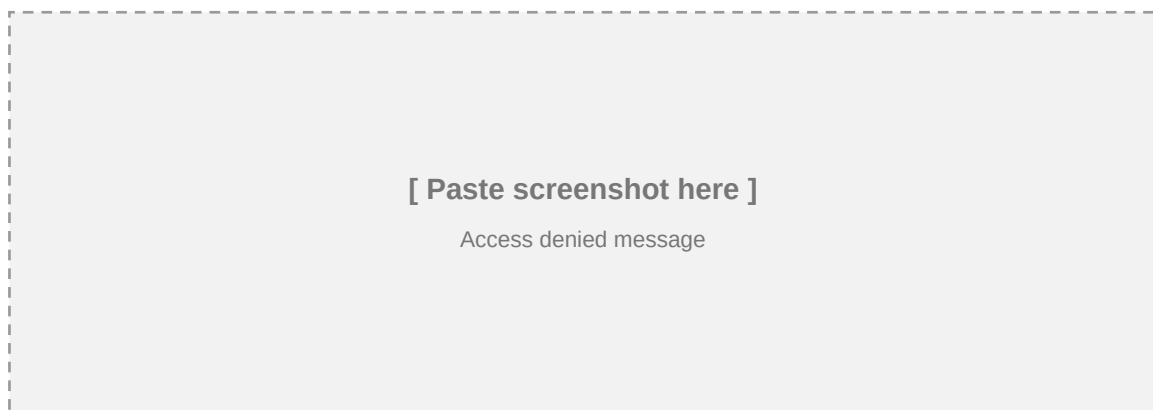


Figure 7.5: ITIL user - security message on direct access

8. ADVANTAGES & DISADVANTAGES

Advantages

- Fine-grained control based on the actual data in a record.
- Native to ServiceNow - no plugins or custom UI required.
- Works consistently for forms, list views, reports and APIs.
- Easy to read, modify and extend; reduces manual effort.
- Strengthens confidentiality and audit/compliance readiness.

Disadvantages

- Poorly written scripts can affect performance or lock users out.
- Requires administrator knowledge and the security_admin elevated role.
- Logic is hard-coded in the script, so rule changes need script edits.
- Needs careful testing because ACLs combine with existing rules.

9. CONCLUSION

The project set out to restrict access to Incident records based on a field value, which standard role-based ACLs cannot do. A script-controlled ACL was designed, configured and tested on a ServiceNow instance. The script evaluates the incident priority together with the user's roles and identity, and grants or denies access accordingly. All planned test cases behaved as expected, so the objectives were achieved. The project also taught the team how ACL evaluation works, how to debug it using impersonation, and why least-privilege design is essential in enterprise platforms.

10. FUTURE SCOPE

- Replace hard-coded values with a custom field (for example *u_restricted*) or system properties.
- Extend the rule to write, delete and field-level ACLs.
- Use groups or roles from a lookup table instead of a fixed role name.
- Add logging and a dashboard of denied access attempts.
- Provide a self-service access-request workflow and custom denial message.
- Apply the same pattern to other tables such as Problem, Change and HR cases.

11. APPENDIX

Source Code (if any)

ACL definition used in the project:

Property	Value
Type	record
Operation	read
Name	incident (table)
Advanced	Checked (script enabled)
Requires role	itil (and incident_manager, admin as per script)

```
// ACL Script: Restrict critical incidents (Priority = 1)
(function() {
    answer = false;

    // 1. Non-critical incident - no extra restriction
    if (current.priority != 1) {
        answer = true;
        return;
    }

    // 2. Critical incident - allow only authorised users
    var userId = gs.getUserID();
    if (gs.hasRole('admin') ||
        gs.hasRole('incident_manager') ||
        current.caller_id == userId ||
        current.assigned_to == userId) {
        answer = true;
    }
})();
```

Dataset Link

No external dataset was used. Sample incident and user records were created manually in the ServiceNow instance for testing.

GitHub & Project Demo Link

GitHub repository: _____

Project demo link: _____

Steps to reproduce: (1) Create role *incident_manager*. (2) Create test users and assign roles. (3) Elevate to *security_admin*. (4) Open System Security > Access Control (ACL) > New. (5) Set Type = record, Operation = read, Name = incident. (6) Tick *Advanced* and paste the script. (7) Save and impersonate each user to test.